

Practice Questions

Banking System Scenario:

Two transactions are running concurrently on a banking database that maintains account balances.

Transaction T1 is responsible for transferring Rs. 10,000 from Account A to Account B.

Transaction T2 is calculating the total sum of balances in both Account A and Account B to generate a financial report.

Here is what happens during execution:

1. T1 starts and reads the balance of Account A (Rs. 50,000).
2. T1 subtracts Rs. 10,000 from Account A (new value: Rs. 40,000), but hasn't updated the database yet.
3. T2 starts and reads the balances of Account A and Account B (Account A = Rs. 50,000, Account B = Rs. 30,000).
4. T2 calculates the total = Rs. 80,000 and writes this to the report.
5. T1 writes Rs. 40,000 to Account A and adds Rs. 10,000 to Account B (Account B = Rs. 40,000).
6. T1 commits.

Question:

Analyze this scenario. What kind of concurrency issue might occur in this case? How can it affect data consistency? Propose a suitable mechanism to handle such situations.

E-Commerce Inventory System:

In an online shopping application, a product has only 1 unit left in stock.

Transaction T1 starts a checkout process for Customer A, who adds the product to their cart.

Simultaneously, Transaction T2 starts a checkout for Customer B, who also adds the same product to their cart.

T1 checks the inventory: quantity = 1.

T2 checks the inventory: quantity = 1.

Both transactions proceed to confirm the order and reduce the stock by 1.

Final stock ends up being -1, even though only one unit was available.

Question:

What kind of issue is demonstrated here? Why is this a problem for inventory management? Suggest ways to prevent this inconsistency.

How to attempt those questions?

Case Study 1: Banking System

Concurrency Problem: Dirty Read (Uncommitted Dependency Problem)

Explanation:

Time	T ₁ (Transfer A → B)	T ₂ (Financial Report)	bal_A	bal_B	Total Reported
t ₁	begin_transaction		50,000	30,000	
t ₂	read(bal_A) → 50,000		50,000	30,000	
t ₃	bal_A = bal_A - 10,000		50,000	30,000	
t ₄		begin_transaction	50,000	30,000	
t ₅		read(bal_A) → 50,000	50,000	30,000	
t ₆		read(bal_B) → 30,000	50,000	30,000	
t ₇		total = 50,000 + 30,000 = 80,000 → write report	50,000	30,000	80,000
t ₈	write(bal_A = 40,000)		40,000	30,000	80,000
t ₉	read(bal_B) → 30,000		40,000	30,000	80,000
t ₁₀	bal_B = bal_B + 10,000		40,000	30,000	80,000
t ₁₁	write(bal_B = 40,000)		40,000	40,000	80,000
t ₁₂	commit		40,000	40,000	80,000

Solution:

To avoid dirty reads, the system should use proper **locking mechanisms (Pessimistic Approach)** to prevent T₂ from accessing the data modified by T₁ until T₁ commits.

Case Study 2: E-Commerce Inventory System

Concurrency Problem: Lost Update

Explanation:

Time	T ₁ (Customer A)	T ₂ (Customer B)	Stock (Quantity)
t ₁	begin_transaction		1
t ₂	read(quantity) → 1		1
t ₃		begin_transaction	1
t ₄		read(quantity) → 1	1
t ₅	quantity = quantity - 1		1
t ₆		quantity = quantity - 1	1
t ₇	write(quantity = 0)		0
t ₈		write(quantity = 0)	-1
t ₉	commit		-1
t ₁₀		commit	-1

Solution:

Solution is using **optimistic concurrency control** with versioning or time stamping to detect conflicting updates and retry the transaction.